A MAS[1] Polymorphic Simulation

Practical Lab for the “Concept Objet” (M2) course

The best: a group of 4, the lower bound: a group of 3

Abstract

Develop a multi-agent system (MAS) simulation in C++, Java, or Rust, featuring four distinct groups of living beings. You may draw inspiration from a heroic fantasy setting (for example, with Orcs, Goblins, Elves, and Humans) or choose another fictional universe, such as Harry Potter, Futurama, Star Wars, The Rings of Power, House of the Dragon, Final Fantasy, or Mario.

> **Scope for adaptation**: The system described below is an example: you are free to substantially adapt both the types of agents and the structure and rules of the simulation, provided that your project meets the requirements set out in the “Expectations” section and correctly applies the object-oriented principles covered in the course.

Your implementation should build on the class inheritance structures and message-passing mechanisms developed in previous practical sessions.

The simulation will involve individuals from the four populations exchanging messages, with interactions now taking into account their positions on a two-dimensional map. All four groups compete from the outset, regardless of any alliances they may form. The winning group is the “wisest”: the one that has collectively gathered the greatest number of distinct messages.

Each group has one special individual called the **Master**, who remains stationary within the group’s SafeZone. A group’s score is the number of distinct messages stored by its Master; messages held only by mobile individuals do not count towards this score. Mobile individuals must therefore collect messages and deliver them to their Master. Interactions take place between individuals occupying adjacent tiles, including diagonally adjacent tiles. Individuals from the same population or alliance may share messages cooperatively, whereas individuals from opposing alliances may obtain messages through confrontation, as specified below.

You will thus develop a multi-agent system that includes all the fundamental object concepts. You must document your project with UML diagrams and identify and use a design pattern. Pseudo-randomness will occur in both the movement of agents on the map and the reception of messages. The following elements can be adapted, but you must still respect the points raised in the section "Expectations" at the end of this document.

[1] https://en.wikipedia.org/wiki/Multi-agent_system

samuel.deleplanque@junia.com

Areas

Let us call the environment in which the four populations of the simulation live "Map." The properties of this map are as follows:

- It is rectangular in shape, with a region called the SafeZone at each corner. Each population has one and only one SafeZone. In proportion, the sum of the four SafeZones should preferably be less than half the map's surface area. Each Master is stationary in its SafeZone, which could be located on a corner of the map, for example.
- The map is discretized into n * m boxes. Each box is identifiable by a horizontal index and a vertical index. A method of the Map class will return the tiles available around a tile given as a parameter: 3 (for a corner of the map), 5 (for a side of the map), or 8 tiles (for the other cases). Another method will return the direction to take to reach your own SafeZone (see subsection on the movement of individuals).
- Obstacles (e.g., trees, furniture, etc.) can be generated randomly at the start of the simulation.
- Each box can contain a maximum of one individual or obstacle; when two individuals interact, they can only do so on two tiles that touch each other (including diagonals).

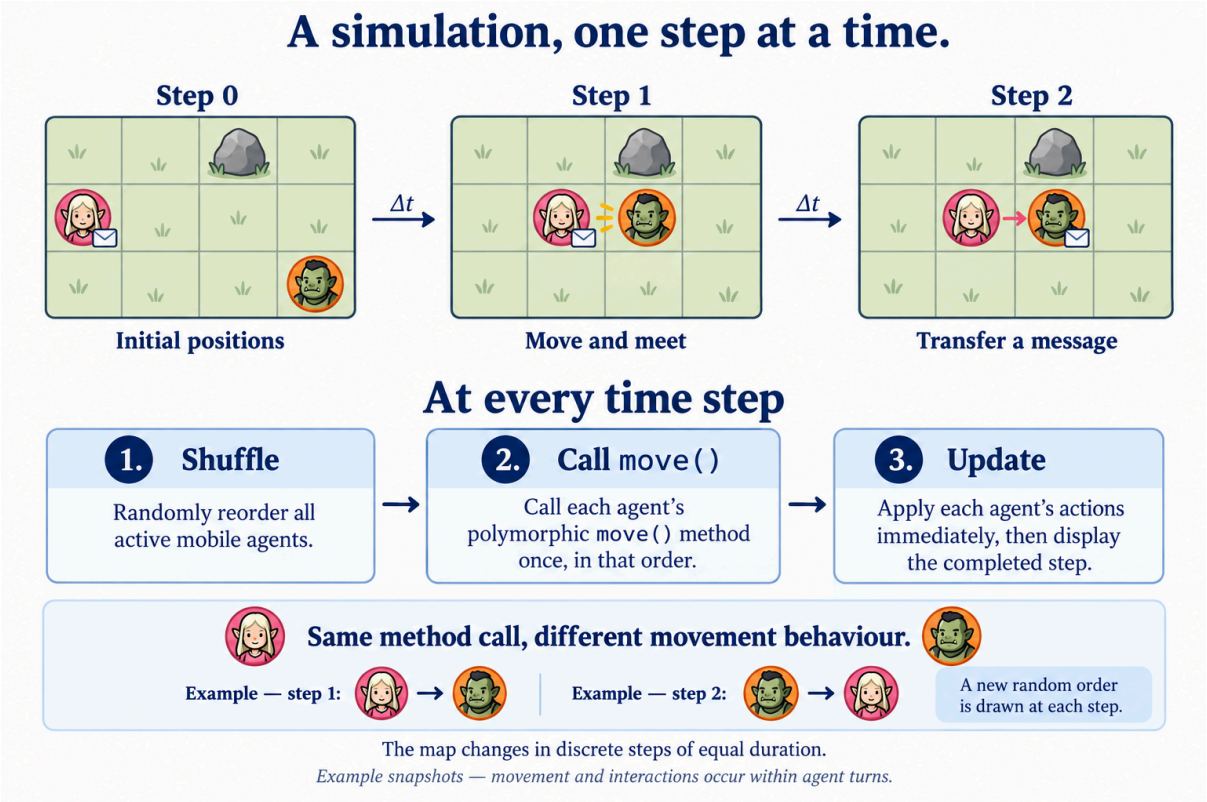
Simulating individuals

The simulation proceeds in discrete steps of equal duration. At the beginning of each step, randomly shuffle the list of mobile individuals. Process them sequentially in this order, allowing each individual that is still active to perform one turn through its move() method. Apply each action immediately so that subsequent individuals act on the updated simulation state. The simulation ends when a Master has collected every distinct message from the initial message set, or when a predefined maximum number of steps has been reached. In the latter case, the group whose Master holds the greatest number of distinct messages wins; equal scores result in a draw. The step limit ensures termination even if some messages become permanently unavailable.

At each time step, active mobile agents are processed in a newly shuffled order. Each agent’s polymorphic move() method is called once, and its actions are applied immediately before the next agent is processed.

At each time step, active mobile agents are processed in a newly shuffled order. Each agent’s polymorphic move() method is called once, and its actions are applied immediately before the next agent is processed.

At each time step, active mobile agents are processed in a newly shuffled order. Each agent’s polymorphic move() method is called once, and its actions are applied immediately before the next agent is processed.



At each time step, active mobile agents are processed in a newly shuffled order. Each agent's polymorphic move() method is called once, and its actions are applied immediately before the next agent is processed.

Orcs, Goblins, Elves and Humans or any other group of 4 populations therefore evolve on the Map. All are living beings, but alliances must be made. For example: Orcs and Goblins generalize as Bad beings while Elves and Humans generalize as Good beings. Consider a class structure reflecting the fundamental object principles by making 2 alliances like in this example. All of these individuals have these attributes in common:

- An array of `strings`
    - for saving first messages as well as those obtained from other groups,
- The last direction taken (North, North-East, East, South-East etc.), in the previous step.
- EP (Energy Points): these points are spent outside the individual's SafeZone and are recovered there. The difficulty is to properly calibrate the starting EPs and their consumption according to the number of tiles traveled in order to obtain a balanced simulation.

The very first design pattern that we will see in class, the singleton, will have to make sense there, with the unique instantiation of MasterTypeGroup' object (eg, for each MasterElf, MasterOrc classes etc.) that we will find as a specialization of a group. If the singleton has not been seen in class yet, it's up to you to document it in the meantime, but you can of course ask the teacher.

Individuals movement rules

- Like the polymorphic `move()` method seen in class and in previous practical work, each step consists of calling this method for all of the individuals. You have to find a

way to generate a different order of calling the individuals `move()` method at each step.

- At each step, and for each individual, the `move()` methods consider several possibilities that you have to define. You can for example use, as in chess, the movement of the Bishop, the Rook, the Queen and the King. It is up to you to define these modes of movement, it can be simpler (e.g., direction and number of boxes to be traversed drawn randomly). The movement can be stopped by an obstacle or an individual standing in the way. Moreover, it is only when the path of a first individual is stopped by a second that there is "Meeting" and the possibility of exchanging messages.
- Each time you move out of the SafeZone, EPs are lost, depending on the number of tiles covered (you must specify these variations). Traveling according to the EP can follow these rules:
    - If EP ≥ 20%, the movements are completely random (number of boxes and/or types of movement),
    - If EP < 20%, the movements are oriented towards the direction of their SafeZone. Once in SafeZone, individuals see their EP increase. It's up to you to calibrate so that people don't go back and forth too much.
    - If EP == 0, the individual will no longer be able to move and becomes an obstacle and therefore loses all his messages.
    - If an individual is blocked by an obstacle, he will have to stop in front of the tile blocking him, but he will lose as many EP as he has left to go. The individual will have saved the last direction taken so that the next movement is not directly blocking.

Encounters between individuals

An individual can encounter another individual on his way[2]. With the 4 types of population, there are several cases:

- Same population (class) encounters:
    - Individuals make the union of all their knowledge[3].
- Different population encounters:
    - Between two individuals who share the same alliance:
        - a random number of messages can be exchanged (up to you to calibrate).
    - Between two individuals who do not share the same alliance:
        - There is confrontation. The winner gets a random number of messages from the loser that the winner does not yet have. The loser loses these messages which are thus transferred. You could implement something simple like “Stone-Leaf-Scissors” or go further: for example, you could develop a turn-based system from role-playing games. The generation of pseudo-random numbers will still be of great help to you here. You are free to add the attributes you want to

[2] For this example, we consider that if the movement ends just in front of a tile with an individual, there is no “share”. For there to be an Encounter, the box of the individual encountered must be part of the planned path.

[3] Be careful not to make any duplicates and check that the messages do not already exist in the table.

your classes, but this on condition that they are well placed in the class hierarchy (inheritance). Strictly no confrontation can take place in a SafeZone since only the individuals of the group of the master who is there can enter it. For others, the outline of SafeZones is characterized by obstacles.

- It is up to you to define the transfer of messages from individuals to the master. For example, you can keep it simple by considering that the master collects them as soon as one of his individuals passes one tile away from him. You are responsible for defining the different behaviors linked to SafeZones.

The following graphic is a representation of a map with the elements described above. Here, 4 teams of 4 different colors compete against each other.



Expectations

- **A discrete simulation with constant time step is expected.** It must set up a multi-agent system (MAS). It is considered here that, during a time step, each individual (except master and those who have become obstacles) tries to move. The order of manipulation of individuals is calculated randomly at each step.
- The **3 fundamental object concepts** must have a central place here. They will also be highlighted in the report. In particular, the methods employed should be **polymorphic** as soon as it is appropriate.
- Use **static attributes** to be able to display the number of instances of each of them.
- The **Singleton design pattern** must find its place in your simulation by reminding you of its usefulness and knowing that it will intervene in 4 different classes.

- The programming language used must be either **Java**, **C++,** or **RUST**. A graphic representation is requested. It can be basic like a console display of the Map with the different protagonists, all with ASCII characters. Display the maximum amount of information per step of simulation, you can resume the method seen in progress to slow down the execution time so that you can understand what is happening at each step. A representation of message transfers will be appreciated. For lack of space, you can simply use letters for these.
- Set up a specific class generating **pseudo-random numbers** for all the needs in the code.
- For the whole group, a **report** in .pdf format, of exactly **6 pages** (“Cover” included), must contain at least one complete **UML class diagram** (with all the attributes and all the methods and reporting their visibility), a **state-machine diagram** (e.g., about the states taken by individuals in a moment of the simulation) and, finally, an **activity diagram** which will represent the important instructions of the “global” algorithm of the simulation (this includes the main loop handling the time step and the loop that manages the movement of individuals).
- Finally, anonymously, explain the roles that have been assigned in your group to develop this project (architecture, code, etc.). Try to define these roles from the first session.

All these expectations must be explained in the report and/or well commented in the code.

To go further (Bonus)

- Events, e.g., meteorites or a virus, can occur randomly on the map. Such events add obstacles until the end of the simulation, or even reach individuals who then turn into obstacles (as if they no longer had EPs outside of SafeZones).
- You can add more complicated movements based on a Shortest Path algorithm in order to avoid obstacles and not stop in front until the next step. For this, we can use a graph model where each node represents a box and each edge represents the possibility of moving from one box to another. Several graphs can be developed depending on the type of move.
- A second Design Pattern can be implemented in addition to the Singleton if it is justified.
- Once the simulation program has been developed, it can be replicated several times and statistics such as the victory percentage of each group can emerge. Be careful not to reset the pseudo-random number generator or at least to do it with a different seed.
- You consider a 3D world and the altitude is then included in the encounter calculation. An individual in a box (x = 1, y = 1, z = 1) will face another individual if the latter encounters these three coordinates during his movement.
- **Advanced bonus: coordinated agents with limited knowledge**
    - (i) Implement a coordinated exploration strategy in which agents make decisions using only their own observations and information received through permitted encounters. Agents must not directly access other agents’ knowledge or the full map state when choosing their actions. In addition to collecting the original messages used for scoring, they may exchange

information about explored areas, obstacles, and their team's objectives. Use this information to coordinate exploration, reduce redundant visits, and decide when an agent should return to its Master. Coordination messages must be clearly distinguished from the original messages and must not contribute to the team's score. All existing movement, energy, encounter, and SafeZone rules still apply.

- (ii) Encapsulate decision-making policies behind a common interface so that the original random policy and the coordinated policy can be substituted without modifying the simulation engine. Compare their performance over repeated runs using the same initial configurations and a documented set of random seeds. Measure at least the number of distinct messages delivered to the Master, the number of agents that become exhausted, and the time required to reach a predefined collection target. Report unsuccessful runs as well as successful ones, and discuss situations in which coordination provides little benefit or performs worse than random exploration.

The code as well as the report must be sent to `samuel.deleplanque@junia.com` by each group referee. The deadline will be one week after the exam day for the same course. Your .zip file should be called CO.SimuTitle.LASTNAME.Firstname.zip (with the LAST NAME and First names of the referrer). The whole team will be named at the start of the report.

Good luck!